

可以。**多教师 OPD (Multi-Teacher On-Policy Distillation, MOPD)** 可以看成把单教师 OPD 里的

$$p_T(\cdot|x, y_{<t})$$

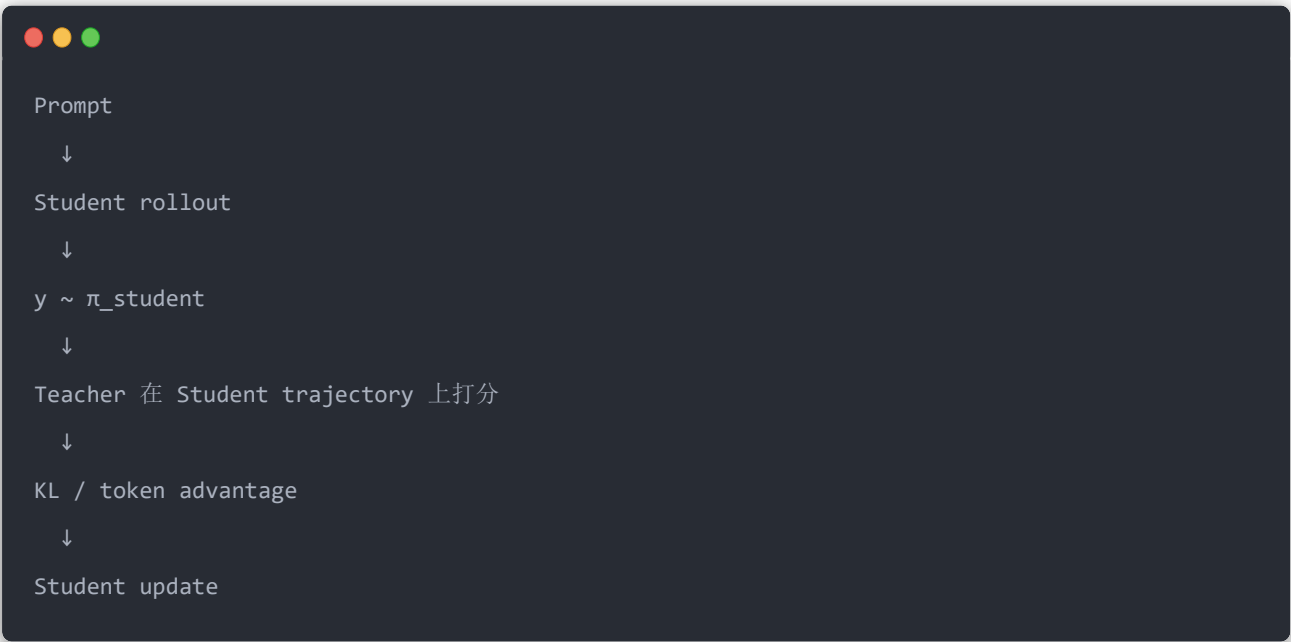
换成多个教师：

$$p_{T_1}, p_{T_2}, \dots, p_{T_K}$$

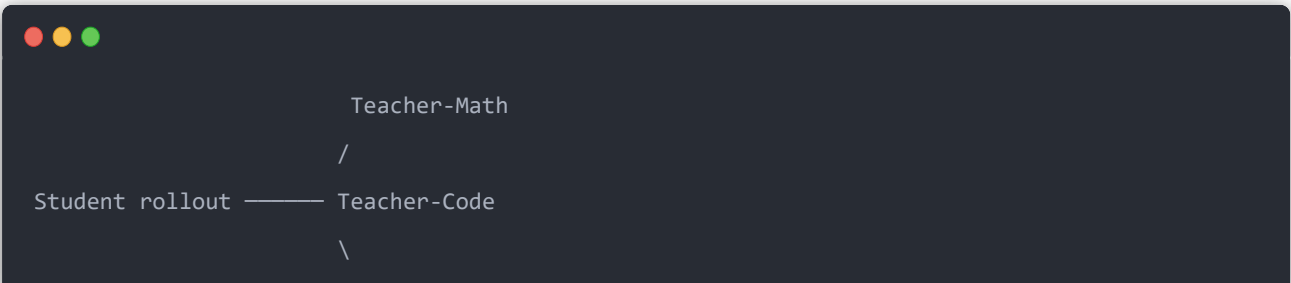
但真正实现时，“多个教师怎么用”其实有几种完全不同的路线。你刚上传的综述把 multi-teacher 当作 OPD signal architecture 的一个重要扩展，而且在工业系统里已经出现了“多个 domain expert → 一个统一 Student”的模式。我先讲理论，再直接看目前能找到的几个开源实现。

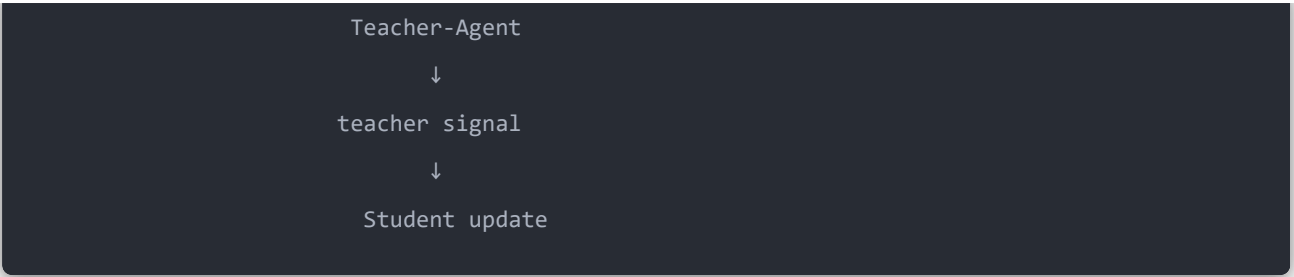
一、多教师 OPD 最基本的训练过程

单教师 OPD 是：



多教师就是：





Student trajectory 依然只由 Student 自己生成:

$$y \sim \pi_{\theta}(\cdot|x)$$

然后多个教师都对同一个 Student prefix:

$$(x, y_{<t})$$

进行评价。

二、最简单的数学形式：多个 Teacher loss 加权

如果有 K 个教师:

$$T_1, T_2, \dots, T_K$$

可以写成:

$$L_{\text{MOPD}} = \mathbb{E}_{y \sim \pi_{\theta}} \sum_t$$

$$\sum_{k=1}^K$$

$w_k D_f(p_{T_k}(\cdot|x, y_{<t}), p_{\theta}(\cdot|x, y_{<t}))$ 满足:

$$w_k \geq 0, \quad \sum_k w_k = 1$$

最简单:

$$w_k = \frac{1}{K}$$

比如 Math Teacher 和 Code Teacher:

$$L = 0.5D(p_S, p_{\text{math}}) + 0.5D(p_S, p_{\text{code}})$$

代码就是：

```
loss_math = reverse_kl(
    student_logits,
    math_teacher_logits,
)

loss_code = reverse_kl(
    student_logits,
    code_teacher_logits,
)

loss = (
    0.5 * loss_math
    + 0.5 * loss_code
)

loss.backward()
```

但真正的大模型训练通常不会这么简单。

三、实际上有三种“多教师 OPD”

这个区别非常重要。

类型	一个 sample 看几个 Teacher	典型形式
Routing MOPD	1 个	Math → Math Teacher
Ensemble MOPD	多个	同一 trajectory 多 Teacher 打分
Debate / Dynamic MOPD	多个	Teacher 先交互，再动态加权

现在开源实现里这三种都有。

四、第一种：Domain Routing MOPD

这是目前**最工程化、最好扩展到 10+ Teacher**的一种方式。假设数据是：

```
{  
  "prompt": "证明这个定理...",  
  "domain": "math"  
}
```

那么：

```
Math sample  
  ↓  
Math Teacher  
  
Code sample  
  ↓  
Code Teacher  
  
Agent sample  
  ↓  
Agent Teacher
```

即：

$$T(x) = \begin{cases} T_{\text{math}}, & x \in \text{Math} \\ T_{\text{code}}, & x \in \text{Code} \\ T_{\text{agent}}, & x \in \text{Agent} \end{cases}$$

loss：

$$L = D_f(p_{T(x)}, p_S)$$

注意：

虽然整个训练有很多 Teacher，但一个 sample 实际只会发给一个 Teacher。这和

“所有 Teacher ensemble” 不是一回事。

五、开源实现 1: KDFlow

目前我觉得 **KDFlow** 是最容易看懂 **Multi-Teacher OPD** 工程实现的项目之一。它已经明确支持：

- off-policy KD
- OPD
- multi-teacher KD
- **multi-teacher OPD**
- SGLang Teacher inference 而且是 2026 年 6 月加入 multi-teacher, 7 月继续更新的。[GitHub](#) 它的配置非常直观：

```
{
  "math": "Qwen3/Qwen3-14B",
  "code": "Qwen3/Qwen3-14B"
}
```

dataset:

```
{
  "conversations": [...],
  "teacher_routing_key": "math"
}
```

KDFlow 明确规定：

每个 sample 根据 `teacher_routing_key` 路由到一个 Teacher。[GitHub](#) 代码逻辑可以抽象为：



```

teachers = {
    "math": math_teacher,
    "code": code_teacher,
}

for batch in dataloader:

    prompts = batch["prompts"]
    routing_keys = batch["teacher_routing_key"]

    # -----
    # 1. Student rollout
    # -----

    rollouts = student.generate(prompts)

    # -----
    # 2. 按 Teacher 分组
    # -----

    teacher_batches = {}

    for i, key in enumerate(routing_keys):
        teacher_batches.setdefault(
            key, []
        ).append(i)

    # -----
    # 3. 不同 Teacher forward
    # -----

    teacher_outputs = {}

    for teacher_name, indices in teacher_batches.items():

        teacher = teachers[teacher_name]

        teacher_outputs[teacher_name] = (
            teacher.forward(
                rollouts[indices]
            )

```

```

    )

    # -----
    # 4. Student forward
    # -----

    student_logits = student(
        rollouts
    ).logits

    # -----
    # 5. 每个 sample 用对应 Teacher
    # -----

    loss = 0

    for key, indices in teacher_batches.items():

        loss += kd_loss(
            student_logits[indices],
            teacher_outputs[key],
        )

    loss.backward()

```

所以其核心不是：

```

for teacher in all_teachers:
    score_every_sample()

```

而是：

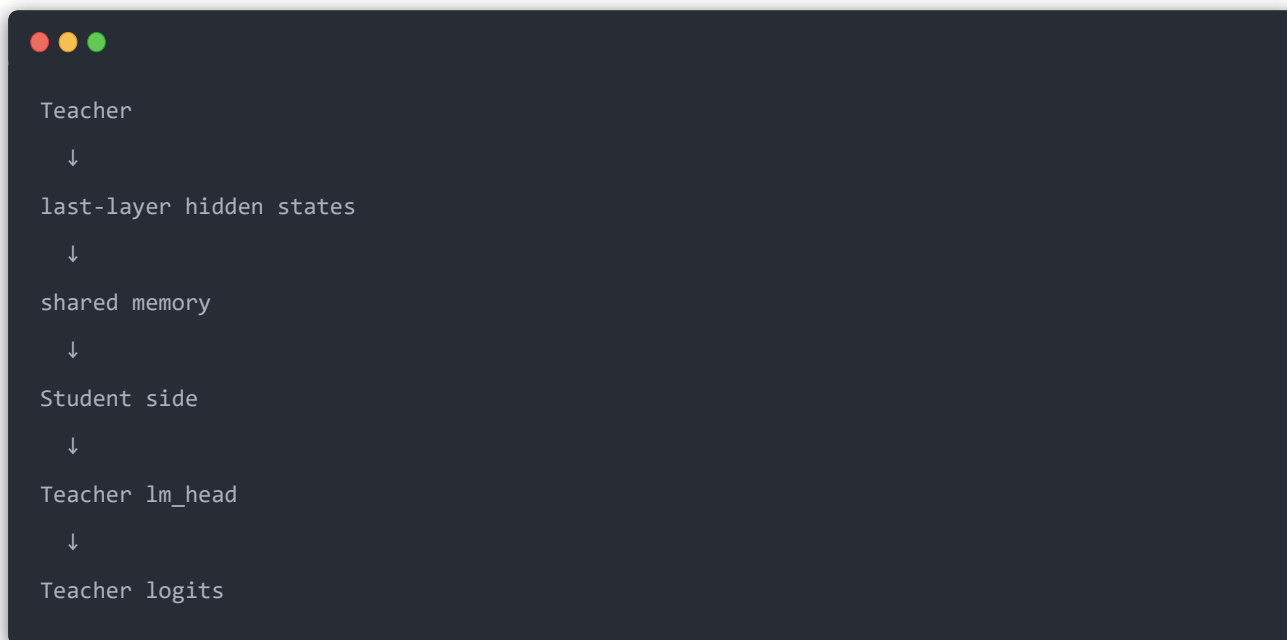
```

teacher = teachers[sample.routing_key]

```

六、KDFlow 的工程实现更有意思

它并不是简单传完整 logits。Teacher 使用 SGLang inference, KDFlow 会：



也就是 Teacher 侧取：

$$h_T \in R^{B \times T \times H}$$

而不是直接传：

$$z_T \in R^{B \times T \times V}$$

然后 Student side 再：

$$z_T = h_T W_{lm_head}^T$$

这样可以显著减少 Teacher→Trainer 的通信。KDFlow 的 README 明确说明它通过共享内存传最后一层 hidden state, 并在 Student 侧应用 Teacher `lm_head`。 [GitHub](#) 这和你之前看到 DeepSeek-V4 survey 里 “避免直接 materialize 多教师 full-vocab logits” 的工程思想是同一个方向。

七、开源实现 2: G-OPD

G-OPD 官方 repo 也已经有 multi-teacher 实现。 目前它只支持两个 domain

Teacher:



```
Math Teacher
```

```
Code Teacher
```

每个样本里：

```
extra_info = {  
    "opd_teacher": "math"  
}
```

或者：

```
extra_info = {  
    "opd_teacher": "code"  
}
```

然后配置里：

```
actor_rollout_ref.ref.model.path=...MathTeacher  
  
actor_rollout_ref.ref.model.base_model_path=...CodeTeacher  
  
actor_rollout_ref.actor.policy_loss.multi_teacher_distill=true
```

[GitHub](#) 所以它实际上也是：

Domain Routing MOPD

而不是两个 Teacher 同时 ensemble。其逻辑大概就是：



```
if sample["opd_teacher"] == "math":  
    teacher = math_teacher  
  
elif sample["opd_teacher"] == "code":  
    teacher = code_teacher
```

然后算 Student sampled-token 上的 OPD signal。

八、这种 Routing 为什么很适合 DeepSeek-V4 类系统？

假设有 10 个专家：

```
Math expert  
Code expert  
Science expert  
Agent expert  
Tool expert  
Reasoning expert  
...
```

一个 Math prompt 根本没必要让：

```
10 个 Teacher
```

全部 forward。否则 Teacher compute：

$$C_{\text{teacher}} \propto K$$

10 个 1T Teacher：

```
Student rollout  
↓  
Teacher 1 forward  
Teacher 2 forward
```

```
...
Teacher 10 forward
```

成本基本不可接受。所以最自然的工业实现就是：

$$x \xrightarrow{\text{router}} T_k$$

于是 Teacher inference 成本仍然近似：

$$O(1)$$

而不是：

$$O(K)$$

这也是为什么目前 NeMo、KDFlow、G-OPD 的工程实现都非常偏向 **routing**。

九、NVIDIA NeMo RL 的 MOPD 更接近工业实现

现在 NeMo RL 已经有专门的：

```
Multi-Teacher On-Policy Distillation
```

实现。它直接构建在 **async GRPO trainer** 上。[GitHub](#) 核心 token advantage 是：

$$\hat{A}_t = \text{sg}[\log \pi_T(y_t|s_t) - \log \pi_S(y_t|s_t)]$$

其中：

- y_t 是 Student rollout 出来的 token;
- π_S 是 rollout 时 Student;
- π_T 是对应 Teacher。NeMo 不需要完整：

$$[V]$$

Teacher distribution。只需要：

$$\log p_T(y_t)$$

即 **Student 实际 sampled token 的 Teacher logprob**。 [GitHub](#) 这点特别重要。

十、NeMo MOPD 为什么很像 GRPO?

它本质上：

```
rollout = student.generate(prompt)

old_logprob = student_logprob(
    rollout.token
)

teacher_logprob = teacher_logprob(
    rollout.token
)

advantage = (
    teacher_logprob
    - old_logprob
).detach()

loss = -(
    advantage
    * student_new_logprob
).mean()
```

也就是把 GRPO:

```
advantage = reward - baseline
```

换成：

```
advantage = teacher_logprob - student_logprob
```

所以：



变成：

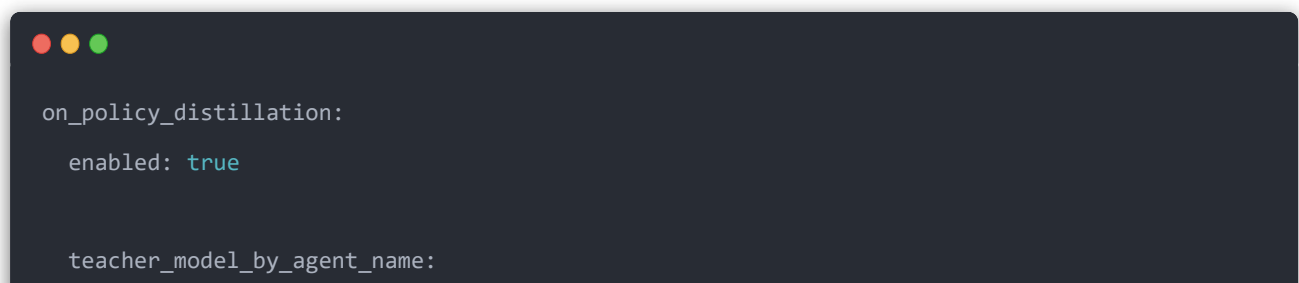


这就是 MiMo-V2-Flash 所说的：

把 knowledge distillation formulation 成 RL process。 [GitHub](#)

十一、NeMo 的多 Teacher 又是怎么选的？

配置：



```
math: /model/math_teacher
code: /model/code_teacher
agent: /model/agent_teacher

default_teacher_alias: math
```

rollout 中会携带:

```
agent_ref
```

然后:

```
teacher_alias = teacher_model_by_agent_name[
    rollout.agent_ref
]
```

接着:

```
teacher.score(rollout)
```

NeMo 文档明确写:

每个 rollout sample 被按 Teacher 分组, 每个 sample **只由一个 Teacher 打分, 不做跨 Teacher ensemble averaging**。 [GitHub](#) 这一点非常关键。 所以:

NeMo MOPD = Multi-Teacher Router

而不是:

$$\sum_k w_k D(p_S, p_{T_k})$$

十二、第二种真正的 Ensemble Multi-Teacher OPD

现在来看真正：

同一个 Student rollout 同时让多个 Teacher 看。公式就是：

$$L =$$

$$\sum_{k=1}^K$$

$w_k D(p_S, p_{T_k})$ 伪代码：

```
rollout = student.generate(prompt)

student_logits = student(rollout).logits

teacher_logits = []

for teacher in teachers:

    with torch.no_grad():

        logits = teacher(
            rollout
        ).logits

        teacher_logits.append(logits)

loss = 0

for weight, logits in zip(
    teacher_weights,
    teacher_logits
):

    loss += weight * reverse_kl(
        student_logits,
        logits
    )

loss.backward()
```

这个才是严格意义上的：

```
Teacher 1
/
Trajectory - Teacher 2
\
Teacher 3
```

十三、开源实现 3: MAD-OPD

这个项目非常适合看这种实现。 MAD-OPD 官方代码里已经有：

```
multi_teacher_jsd_loss(...)
```

核心就是：

```
for i, (teacher_logits, weight) in enumerate(
    zip(
        teacher_logits_list,
        teacher_weights
    )
):

    loss_i = generalized_jsd_loss_single(
        student_logits=student_logits,
        teacher_logits=teacher_logits,
        ...
    )

    total_loss += weight * loss_i
```

它明确实现：

$$L = \sum_i w_i D(p_S, p_{T_i})$$

默认没有权重的话：

$$w_i = \frac{1}{K}$$

[GitHub](#) 所以这和 KDFlow/NeMo 的 routing 是完全不同的。

十四、MAD-OPD 更进一步：Teacher 权重不是固定的

它做了一个很有意思的东西：

$$w_k = \text{softmax}\left(\frac{c_k}{\tau}\right)$$

其中：

$$c_k$$

是 Teacher confidence。源码中就是：

```
def _softmax_confidence_weights(  
    score_a,  
    score_b,  
    tau_conf=1.0  
):  
  
    logits = [  
        score_a / 100.0 / tau_conf,  
        score_b / 100.0 / tau_conf  
    ]  
  
    ...
```

所以：

```
Teacher A confidence = 90  
Teacher B confidence = 60
```

会得到：



```
Teacher A weight > Teacher B weight
```

而不是固定：



```
[0.5, 0.5]
```

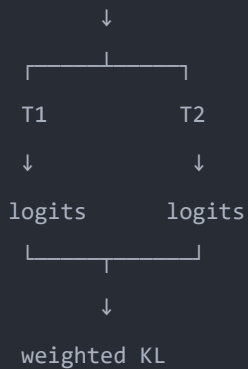
[GitHub](#)

十五、MAD-OPD 最特殊的是 Teacher 先“辩论”

普通 Ensemble：



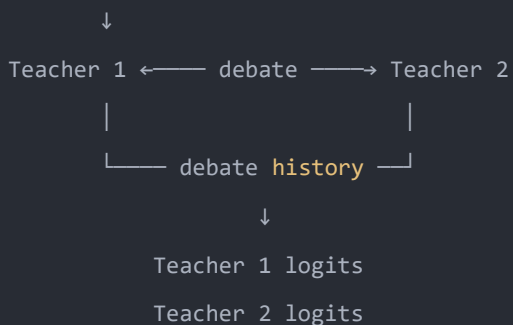
```
Student trajectory
```



MAD-OPD：



```
Student trajectory
```



↓
confidence weighted KL
↓
Student

论文/代码说明：

$$\mathcal{H}_m^R$$

即多轮 Teacher debate history，只让 Teacher 看到。Teacher 最终看的 context 是：

$$x + \mathcal{H}_m^R + y_{<t}^S$$

Student 则只看：

$$x + y_{<t}^S$$

因此产生一个类似 privileged-information 的 gap。 [GitHub](#)

十六、它是怎么拿 Teacher logits 的？

这里代码特别值得看。Student 已经 rollout：

```
y_student
```

Teacher **不能自己 generate 新答案**。否则就又变成 Teacher trajectory 了。所以 MAD-OPD 使用：

force decode Student output 流程：

```
Student:  
prompt → y1 y2 y3 y4
```

Teacher:

```
prompt + debate history
+
student y1 y2 y3 y4

↓ force decode

每个位置输出 Teacher logits
```

代码 sidecar 做的是：

```
# 1. teacher context
context = prompt + debate_history

# 2. append Student response
tokens = context + student_output

# 3. teacher forward
teacher_logits = teacher(tokens)

# 4. 取 Student response 位置
teacher_logits = teacher_logits[
    student_response_positions
]
```

返回：

$$[1, T, V]$$

完整 Teacher logits。 [GitHub](#) 这其实就是 OPD 的标准 teacher scoring。

十七、两个 Teacher 的实际 GPU pipeline

MAD-OPD repo 给了一个很清楚的部署：

```
GPU 0
├─ vLLM Teacher-1
```

```
└─ Force-decode Sidecar-1

GPU 1
└─ vLLM Teacher-2
└─ Force-decode Sidecar-2

GPU 2..N-1
└─ Student training
```

[GitHub](#) 所以一条 Student trajectory:

```
GPU0
Teacher 1
force decode
/
Student rollout
\
Teacher 2
force decode
GPU1

↓
logits1 + logits2
↓
Student GPUs
↓
loss
```

这就是一个非常标准的 **真正 multi-teacher ensemble OPD** 实现。

十八、它的 loss 代码可以压缩成这样

源码本质上是：

```
teacher_weights = [
    alpha,
    beta,
]
```

```
loss_teacher_1 = divergence(
    student_logits,
    teacher_1_logits,
)

loss_teacher_2 = divergence(
    student_logits,
    teacher_2_logits,
)

loss = (
    alpha * loss_teacher_1
    +
    beta * loss_teacher_2
)
```

如果没有 dynamic weighting:

```
alpha = 0.5
beta = 0.5
```

MAD:

```
alpha, beta = confidence_softmax(
    teacher1_score,
    teacher2_score,
)
```

然后:

```
loss.backward()
```

代码明确说明每个 Teacher 的 divergence 独立计算，再由动态权重组合。 [GitHub](#)

十九、一个很容易搞错的地方：不是先平均 logits

你可能会自然想到：

```
teacher_logits = (  
    logits_t1  
    +  
    logits_t2  
) / 2
```

然后：

```
KL(student, teacher_logits)
```

但这**通常不等价于**：

$$\frac{1}{2}KL(p_S, p_{T_1}) + \frac{1}{2}KL(p_S, p_{T_2})$$

因为：

$$D\left(p_S, \sum_k w_k p_{T_k}\right) \neq \sum_k w_k D(p_S, p_{T_k})$$

MAD-OPD 的实现采用的是后者：

```
loss = 0  
  
for teacher:  
    loss += weight * divergence(  
        student,  
        teacher  
    )
```

而不是简单 logits average。 [GitHub](#)

二十、Routing 和 Ensemble 到底哪个好？

这取决于你的 Teacher 定义。假如：

```
Math Teacher
Code Teacher
Agent Teacher
```

领域互相比较独立，最合理是：

Routing

因为：

```
Math prompt
↓
Math Teacher
```

没必要让 Code Teacher 参与。这也是：

- KDFlow
- G-OPD
- NeMo RL 目前采用的主要方式。 [GitHub+1](#) 但如果：

```
Teacher A = Qwen3-32B
Teacher B = Qwen3-30B-A3B
```

两个都是同领域但具有不同 reasoning pattern，那么：

Ensemble

可能更合理。这就是 MAD-OPD 这种方法。

二十一、再看 MiMo-V2-Flash

MiMo-V2-Flash 的技术说明明确说使用：

Multi-Teacher On-Policy Distillation 并指出：

- Student 从自己的 responses 学；
- domain-specific expert teachers 提供 token-level dense guidance；
- 整个 KD 被建模为 RL process。 [GitHub](#) 结合 NVIDIA NeMo RL 当前 MOPD 实现，可以看到其工程形态非常明确：

```
Student rollout
  ↓
Domain / Agent routing
  ↓
Corresponding Teacher
  ↓
Teacher sampled-token logprob
  ↓
 $A_t = \log p_T - \log p_S$ 
  ↓
policy gradient
```

注意这里我把“NeMo 实现”和“MiMo 报告”分开说：[NeMo 文档](#)明确是每个 sample exactly one teacher，不做 ensemble；MiMo README 本身没有在这段公开说明里把内部 router/ensemble 细节全部展开。 [GitHub+1](#)

二十二、所以 Multi-Teacher OPD 可以写成两套完全不同的代码

版本 A：工业上更常见的 Router MOPD

```
rollouts = student.generate(prompts)

for domain in domains:

    idx = routing_key == domain
```

```

teacher = teachers[domain]

teacher_logprob = teacher.score(
    rollouts[idx]
)

student_logprob = student.score(
    rollouts[idx]
)

advantage = (
    teacher_logprob
    -
    student_logprob.detach()
)

loss += policy_gradient_loss(
    student_logprob,
    advantage,
)

```

计算量：

$$\approx 1 \times Teacher$$

每个 sample。

版本 B: 真正 Ensemble MOPD

```

rollouts = student.generate(prompts)

student_logits = student(
    rollouts
).logits

teacher_logits_list = []

for teacher in teachers:

    with torch.no_grad():

```

```
logits = teacher(
    rollouts
).logits

teacher_logits_list.append(logits)

loss = 0

for w, teacher_logits in zip(
    weights,
    teacher_logits_list
):

    loss += w * divergence(
        student_logits,
        teacher_logits
    )

loss.backward()
```

计算量：

$$\approx K \times Teacher$$

每个 sample。

二十三、我建议你重点看这三个开源项目

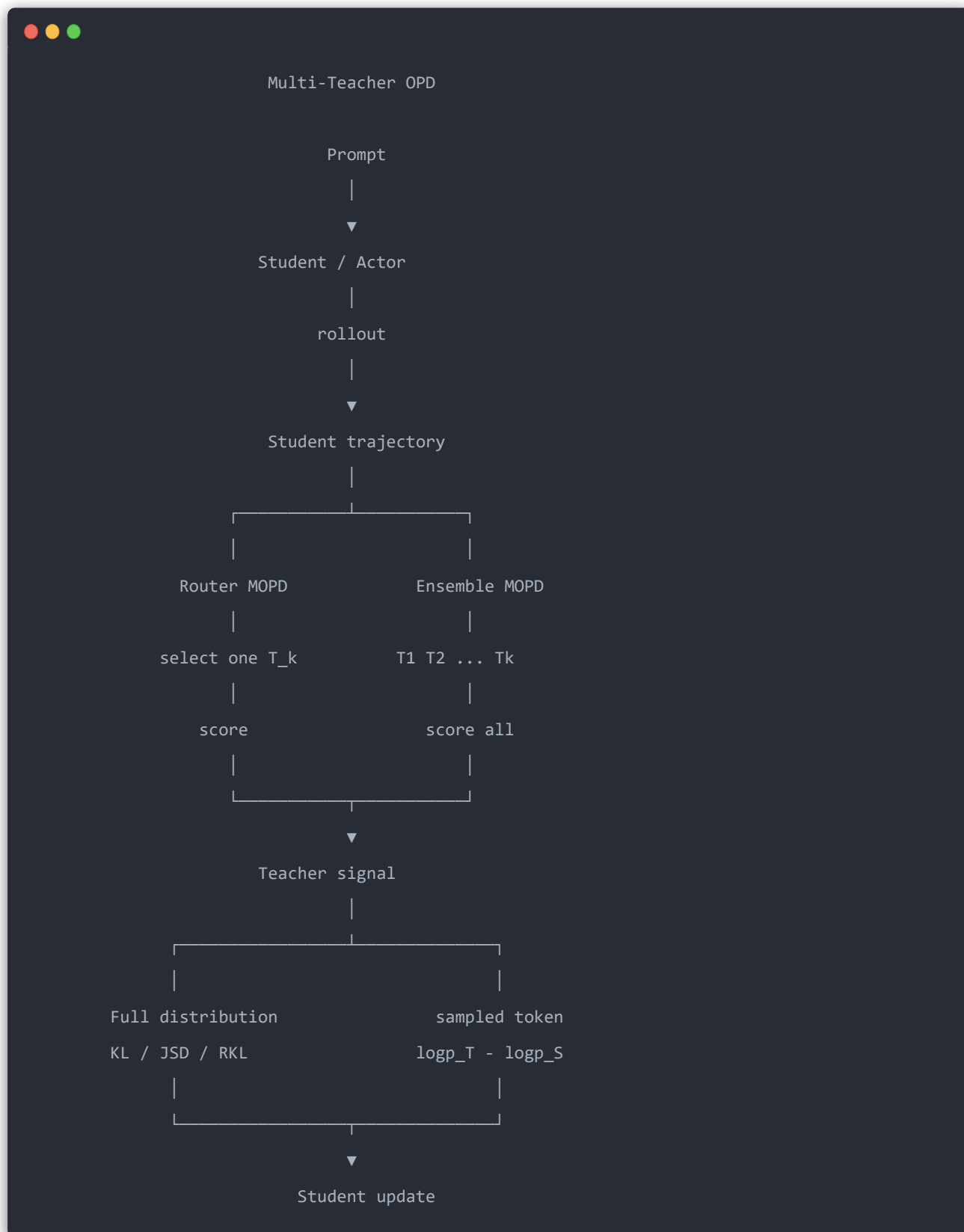
按“能学到什么”排序：

项目	多教师方式	最值得看的地方
KDFlow	Domain routing	SGLang + hidden-state transfer + router
NVIDIA NeMo RL	Agent/domain routing	把 MOPD 直接实现成 GRPO-style token advantage
MAD-OPD	多 Teacher ensemble + debate	真正多 Teacher 同时 score, 一个 Student loss

G-OPD 也值得看，但目前 repo 的 multi-teacher 只支持两个 Teacher，而且主要也是 domain routing。 [GitHub](#)

二十四、如果你熟悉的 RL 系统语言来理解

我建议直接记成：



所以 **MOPD 并不等于“多个 Teacher logits 平均一下”**。真正要先回答三个问题：

1. **Teacher 怎么选？** Router 还是全部参与。
2. **Teacher signal 怎么融合？** loss average、confidence weighting、debate，还是 sampled-token advantage。
3. **Teacher 输出什么？** full-vocab logits、top-k logits、hidden states，还是 Student sampled token 的 logprob。这三个选择基本就决定了 MOPD 的系统架构。如果是为了真正复现，我会优先建议你从 **KDFlow 的 multi-teacher on-policy example** 下手，因为它已经是 **SGLang rollout/teacher + distributed student training** 的完整工程，而不是只有论文伪代码；然后再看 **NeMo RL 的 MOPD**，你会非常容易把它和 GRPO 的 rollout → logprob → advantage → update 一一对应起来。